

benchmarking tools

Qun Li

15 Aug 2025 at 14:26:31

Contents

1	ENV	1
2	Libraries	2
3	Setting	2
4	loadData ENCODE	3
5	cSCC	6
6	cSCC mutation signature	10
7	Mouse scATAC	13
8	Mouse scRNA	17
9	Mouse scRNA scATA calConsistencyscore	20
10	BrainPFC	22
11	sessionInfo	23

Reading WGS data and performing the bootstrap procedure may require substantial memory and computational time

1 ENV

```
rm(list = ls())  
setwd("/Users/liqun/Desktop/scMutrace-tutorial/ReproducibilityAnalysis/Figure2/")
```

2 Libraries

```
library(ggsci)
library(dplyr)
##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
library(scales)
library(readxl)
library(forcats)
library(ggplot2)
library(reshape2)
library(pheatmap)
library(circlize)
## =====
## circlize version 0.4.16
## CRAN page: https://cran.r-project.org/package=circlize
## Github page: https://github.com/jokergoo/circlize
## Documentation: https://jokergoo.github.io/circlize\_book/book/
##
## If you use it in published research, please cite:
## Gu, Z. circlize implements and enhances circular visualization
##   in R. Bioinformatics 2014.
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(circlize))
## =====
library(patchwork)
library(gridExtra)
##
## Attaching package: 'gridExtra'
## The following object is masked from 'package:dplyr':
##
##   combine
library(bedtoolsr)
## Warning in fun(libname, pkgname): bedtoolsr was built with bedtools version 2.30.0 but you have vers
## options(bedtools.path = "[bedtools path]")
library(UpSetR)
```

3 Setting

```
sample_size <- 0.6
iteration_times <- 50
```

4 loadData ENCODE

ENCDO451RUA transverse colon tissue Info: [https://www.encodeproject.org/matrix/?type=Experiment&searchTerm=ENCDO451RUA&assay_title=snATAC-seq&assay_title=WGS_snATAC_\(ENCFF491HQL\)](https://www.encodeproject.org/matrix/?type=Experiment&searchTerm=ENCDO451RUA&assay_title=snATAC-seq&assay_title=WGS_snATAC_(ENCFF491HQL)): <https://www.encodeproject.org/experiments/ENCSR349XKD/> WGS (ENCFF907ASL): <https://www.encodeproject.org/experiments/ENCSR613XEH/>

ENCDO845WKR transverse colon tissue Info: [https://www.encodeproject.org/matrix/?type=Experiment&searchTerm=ENCDO845WKR&assay_title=snATAC-seq&assay_title=WGS_snATAC_\(ENCFF354SCV\)](https://www.encodeproject.org/matrix/?type=Experiment&searchTerm=ENCDO845WKR&assay_title=snATAC-seq&assay_title=WGS_snATAC_(ENCFF354SCV)): <https://www.encodeproject.org/experiments/ENCSR434SXE/> WGS (ENCFF501WAL): <https://www.encodeproject.org/experiments/ENCSR410ALS/>

ENCDO793LXB transverse colon tissue Info: [https://www.encodeproject.org/matrix/?type=Experiment&searchTerm=ENCDO793LXB&assay_title=snATAC-seq&assay_title=WGS_snATAC_\(ENCFF692LTO\)](https://www.encodeproject.org/matrix/?type=Experiment&searchTerm=ENCDO793LXB&assay_title=snATAC-seq&assay_title=WGS_snATAC_(ENCFF692LTO)): <https://www.encodeproject.org/experiments/ENCSR997YNO/> WGS (ENCFF926UDZ): <https://www.encodeproject.org/experiments/ENCSR997HAI/>

Reading WGS data and performing the bootstrap procedure may require substantial memory and computation
scATAC

```
path_encode_excel <- "../Data/HumanColon/GermlineMutation_results_HumanColon_sup.xlsx"
excel_sheets(path_encode_excel)
## [1] "freebayes" "Monopogen" "SAMtools" "scAllele" "scMutrace" "Strelka2"
## [7] "VarScan2"
```

init res

```
col_benchmarking <- c("sampleID", "Iteration", "Tool", "Precision", "Recall", "F1Score")
SampleIDs_HumanColon <- c("ENCDO451RUA", "ENCDO793LXB", "ENCDO845WKR", "ENCDO271OUW")
Tools_benchmarking_germline <- c("freebayes", "Monopogen", "SAMtools", "scAllele",
                                "scMutrace", "Strelka2", "VarScan2")
```

```
data_benchmarking_humanColon_res <- as.data.frame(matrix(data = 0, nrow = length(SampleIDs_HumanColon) ,
colnames(data_benchmarking_humanColon_res) <- col_benchmarking
```

generate benchmarking data

WGS

```
HumanColon_WGS <- readRDS("../Data/HumanColon/GermlineMutation_results_HumanColon_WGS.rds")
```

```
HumanColon_WGS_benchmarking <- list()
```

```
for (sampleID in SampleIDs_HumanColon){
  data_wgs <- HumanColon_WGS[[sampleID]]
  colnames(data_wgs) <- c("chrom_wgs", "pos_start_wgs", "pos_end_wgs", "ref_wgs", "alt_wgs" )
  HumanColon_WGS_benchmarking[[sampleID]] <- data_wgs
}
```

```
HumanColon_RNA_benchmarking <- list()
```

```
for (sampleID in SampleIDs_HumanColon){
  data_rna_ini <- data.frame()
  for (tool in Tools_benchmarking_germline){
    data_rna_total <- read_excel(path_encode_excel, sheet = tool) %>% filter(SampleID == sampleID)
    data_rna_ini <- rbind(data_rna_ini, data_rna_total)
  }
  colnames(data_rna_ini) <- c("chrom_rna", "pos_start_rna", "pos_end_rna",
                             "ref_rna", "alt_rna", "Type_rna", "Method_rna",
                             "SampleID", "Genome", "SequencingType_rna")
  HumanColon_RNA_benchmarking[[sampleID]] <- data_rna_ini
}
```

```

HumanColon_Data_benchmarking <- list()
for (sampleID in SampleIDs_HumanColon){
  data_RNA_WGS <- data.frame()
  data_WGS <- HumanColon_WGS_benchmarking[[sampleID]]
  data_WGS$id <- paste(data_WGS$chrom_wgs, data_WGS$pos_start_wgs, data_WGS$ref_wgs, sep = "_")
  for (tool in Tools_benchmarking_germline){
    data_RNA <- HumanColon_RNA_benchmarking[[sampleID]] %>% filter(Method_rna == tool)
    data_RNA$id <- paste(data_RNA$chrom_rna, data_RNA$pos_start_rna, data_RNA$ref_rna, sep = "_")
    data_RNA_WGS_tool <- merge(data_RNA, data_WGS, by = "id", all = T)
    data_RNA_WGS_tool$Method_rna <- tool
    data_RNA_WGS <- rbind(data_RNA_WGS, data_RNA_WGS_tool)
  }
  HumanColon_Data_benchmarking[[sampleID]] <- data_RNA_WGS
}

# bootstrap
set.seed(2025)
line_num <- 0
for (iter in 1:iteration_times){
  for (sampleID in SampleIDs_HumanColon){
    HumanColon_Data_benchmarking_bm <- HumanColon_Data_benchmarking[[sampleID]] %>% sample_frac(sample_
    for (tool in Tools_benchmarking_germline){
      line_num <- line_num + 1
      HumanColon_Data_benchmarking_bm_tool <- HumanColon_Data_benchmarking_bm %>% filter(Method_rna == 
      total_calls <- sum(!is.na(HumanColon_Data_benchmarking_bm_tool$chrom_rna))
      total_truth <- sum(!is.na(HumanColon_Data_benchmarking_bm_tool$chrom_wgs))
      tp <- nrow(na.omit(HumanColon_Data_benchmarking_bm_tool))
      fp <- total_calls - tp
      fn <- total_truth - tp
      sensitivity <- ifelse(total_truth > 0, tp / (tp + fn), NA)
      precision <- ifelse(total_calls > 0, tp / (tp + fp), NA)
      f1 <- ifelse(!is.na(sensitivity) & !is.na(precision) &
                    (sensitivity + precision) > 0,
                    2 * (precision * sensitivity) / (precision + sensitivity),
                    NA)

      data_benchmarking_humanColon_res[line_num, "sampleID"] <- sampleID
      data_benchmarking_humanColon_res[line_num, "Iteration"] <- iter
      data_benchmarking_humanColon_res[line_num, "Tool"] <- tool
      data_benchmarking_humanColon_res[line_num, "Precision"] <- precision
      data_benchmarking_humanColon_res[line_num, "Recall"] <- sensitivity
      data_benchmarking_humanColon_res[line_num, "F1Score"] <- f1
    }
  }
}

colorAssign_0 = c(freebayes = "#FFDC91FF", SAMtools = "#7876B1FF",
                  scAllele = "#20854EFF", VarScan2 = "#0072B5FF",
                  Strelka2= "#9B9898", Monopogen = "#6F99ADFF",
                  scMutrace = "#BC3C29FF")

data_benchmarking_humanColon_res <- data_benchmarking_humanColon_res %>%
  mutate(Tool = fct_reorder(Tool, F1Score, .fun = mean, .na_rm = TRUE))

```

```

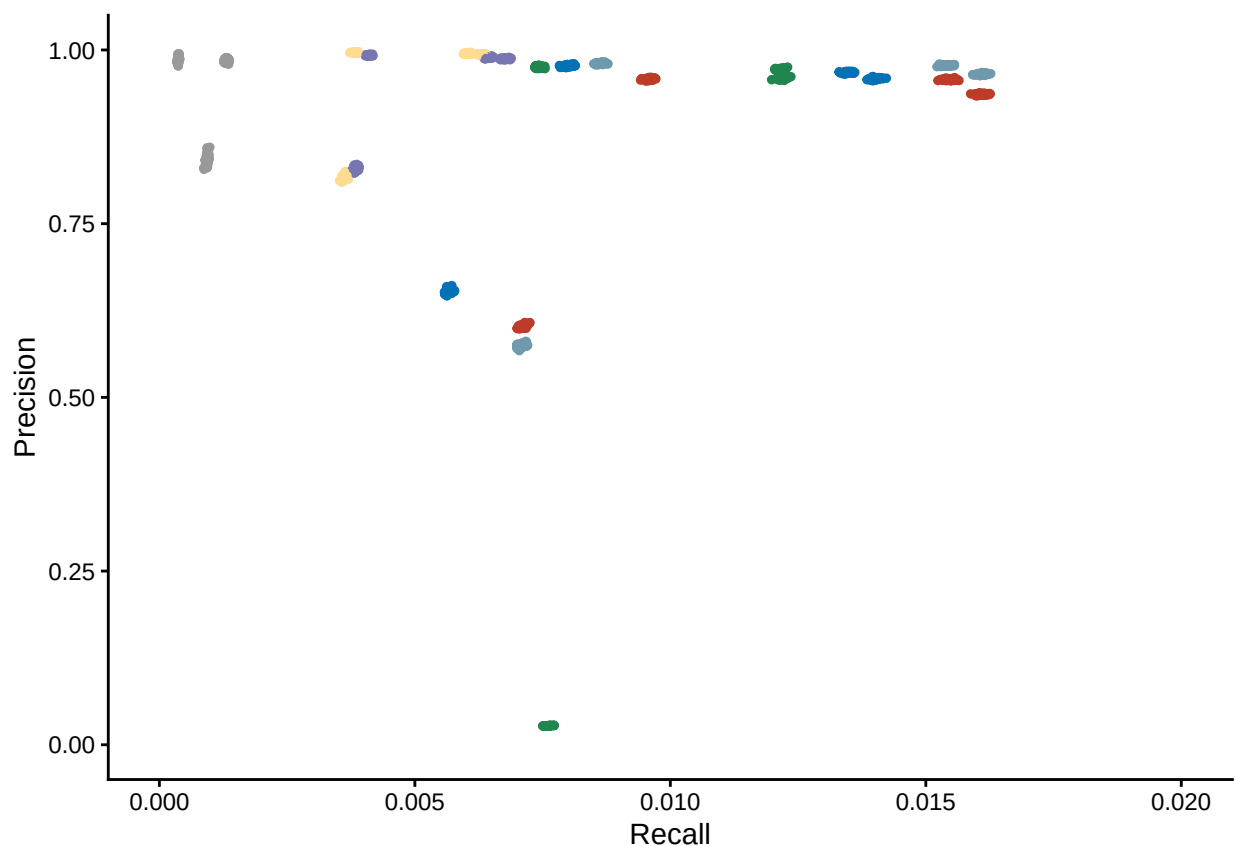
data_benchmarking_humanColon_res <- na.omit(data_benchmarking_humanColon_res)

p_humanColon_point <- ggplot(data = data_benchmarking_humanColon_res) +
  geom_point(mapping = aes(x = Recall, y = Precision, color = Tool), size = 1, show.legend = FALSE) +
  scale_color_manual(values = colorAssign_0) +
  xlab("Recall") + ylab("Precision") +
  xlim(0, 0.02) +
  ylim(0, 1) +
  theme_classic()

p_humanColon_box <- ggplot(data = data_benchmarking_humanColon_res, aes(x = Tool, y = F1Score)) +
  stat_summary(mapping=aes(fill = Tool),fun=mean, geom = "bar", fun.args = list(mult=1), width=0.7) +
  stat_summary(fun.data=mean_sdl,fun.args = list(mult=1), geom="errorbar",width=0.2) +
  scale_fill_manual(values = colorAssign_0) +
  labs(x = "",y = "F-score") +
  scale_x_discrete(guide = guide_axis(angle = 45)) +
  theme_classic() + guides(fill=guide_legend(title="Method"))

p_humanColon_point

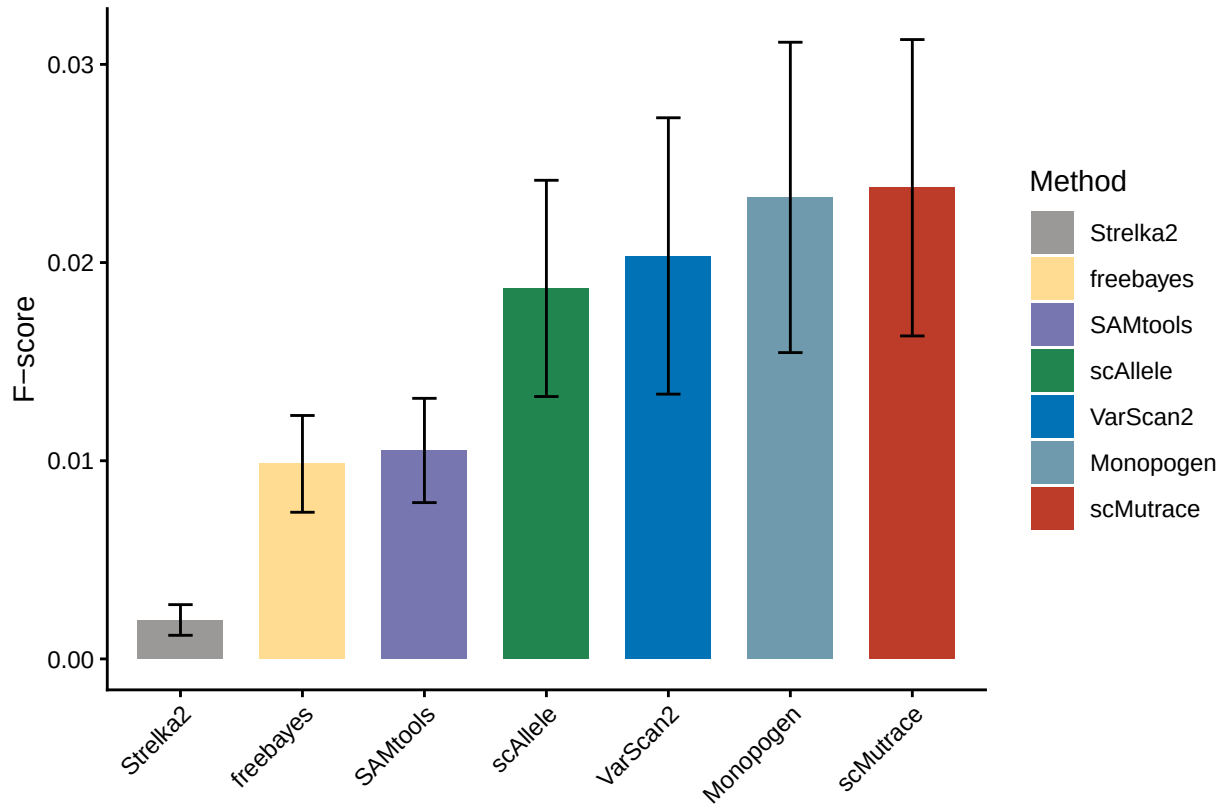
```



```

p_humanColon_box

```



5 cSCC

```

path_cSCC_excel <- "../Data/HumancSCC/SomaticMutation_results_cSCC_sup.xlsx"
excel_sheets(path_cSCC_excel)
## [1] "WES"          "freeBayes" "Monopogen" "SAMtools" "scAllele" "scMutrace"
## [7] "SComatic" "Strelka2" "VarScan2" "SBS"

Tools_benchmarking_cSCC <- c("freeBayes", "Monopogen", "SAMtools", "scAllele", "scMutrace", "SComatic",
SampleIDs_cSCC <- c("P2", "P3", "P4", "P5", "P6", "P7", "P8", "P9", "P10")

col_benchmarking_cSCC <- c("Iteration", "Tool", "Precision", "Recall", "F1Score")

data_benchmarking_cSCC_res <- as.data.frame(matrix(data = 0, nrow = iteration_times * length(Tools_benchmarking_cSCC),
ncol = length(col_benchmarking_cSCC)))

colnames(data_benchmarking_cSCC_res) <- col_benchmarking_cSCC

cSCC_WES <- read_excel(path_cSCC_excel, sheet = "WES")

cSCC_WES$id <- paste(cSCC_WES$chrom, cSCC_WES$pos_start, cSCC_WES$ref, cSCC_WES$SampleID, sep = "_")
colnames(cSCC_WES) <- c("Project_WES", "Sample_WES", "ID_WES", "Genome_WES",
"mut_type_WES", "chrom_WES", "pos_start_WES", "pos_end_WES",

```

```

      "ref_WES", "alt_WES", "Type_WES", "Method_WES", "SampleID_WES",
      "id" )

cSCC_Data_benchmarking <- list()

for (tool in Tools_benchmarking_cSCC){
  cSCC_RNA <- read_excel(path_cSCC_excel, sheet = tool)
  cSCC_RNA$id <- paste(cSCC_RNA$chrom, cSCC_RNA$pos_start, cSCC_RNA$ref, cSCC_RNA$SampleID, sep = "_")
  colnames(cSCC_RNA) <- c("Project_RNA", "Sample_RNA", "ID_RNA", "Genome_RNA",
    "mut_type_RNA", "chrom_RNA", "pos_start_RNA", "pos_end_RNA",
    "ref_RNA", "alt_RNA", "Type_RNA", "Method_RNA", "SampleID_RNA",
    "id" )
  cSCC_WES_RNA_tool <- merge(cSCC_WES, cSCC_RNA, by = "id", all = T)
  cSCC_WES_RNA_tool$Method_RNA <- tool
  cSCC_Data_benchmarking[[tool]] <- cSCC_WES_RNA_tool
}

set.seed(2025)
line_num <- 0
for (iter in 1:iteration_times){
  for (tool in Tools_benchmarking_cSCC){
    line_num <- line_num + 1
    cSCC_Data_benchmarking_bm <- cSCC_Data_benchmarking[[tool]] %>% sample_frac(sample_size)
    cSCC_Data_benchmarking_bm_tool <- cSCC_Data_benchmarking_bm %>% filter(Method_RNA == tool)
    total_calls <- sum(!is.na(cSCC_Data_benchmarking_bm_tool$chrom_RNA))
    total_truth <- sum(!is.na(cSCC_Data_benchmarking_bm$chrom_WES))
    tp <- nrow(na.omit(cSCC_Data_benchmarking_bm_tool))
    fp <- total_calls - tp
    fn <- total_truth - tp
    sensitivity <- ifelse(total_truth > 0, tp / (tp + fn), NA)
    precision <- ifelse(total_calls > 0, tp / (tp + fp), NA)
    f1 <- ifelse(!is.na(sensitivity) & !is.na(precision) &
      (sensitivity + precision) > 0,
      2 * (precision * sensitivity) / (precision + sensitivity),
      NA)
    data_benchmarking_cSCC_res[line_num, "Iteration"] <- iter
    data_benchmarking_cSCC_res[line_num, "Tool"] <- tool
    data_benchmarking_cSCC_res[line_num, "Precision"] <- precision
    data_benchmarking_cSCC_res[line_num, "Recall"] <- sensitivity
    data_benchmarking_cSCC_res[line_num, "F1Score"] <- f1
  }
}

colorAssign_1 = c(freeBayes = "#FFDC91FF", SAMtools = "#7876B1FF",
  scAllele = "#20854EFF", VarScan2 = "#0072B5FF",
  Strelka2 = "#9B9898", SComatic = "#fb9a99",
  Monopogen = "#6F99ADFF", scMutrace = "#BC3C29FF")

data_benchmarking_cSCC_res <- data_benchmarking_cSCC_res %>%
  mutate(Tool = fct_reorder(Tool, F1Score, .fun = mean, .na_rm = TRUE))

data_benchmarking_cSCC_res <- na.omit(data_benchmarking_cSCC_res)

```

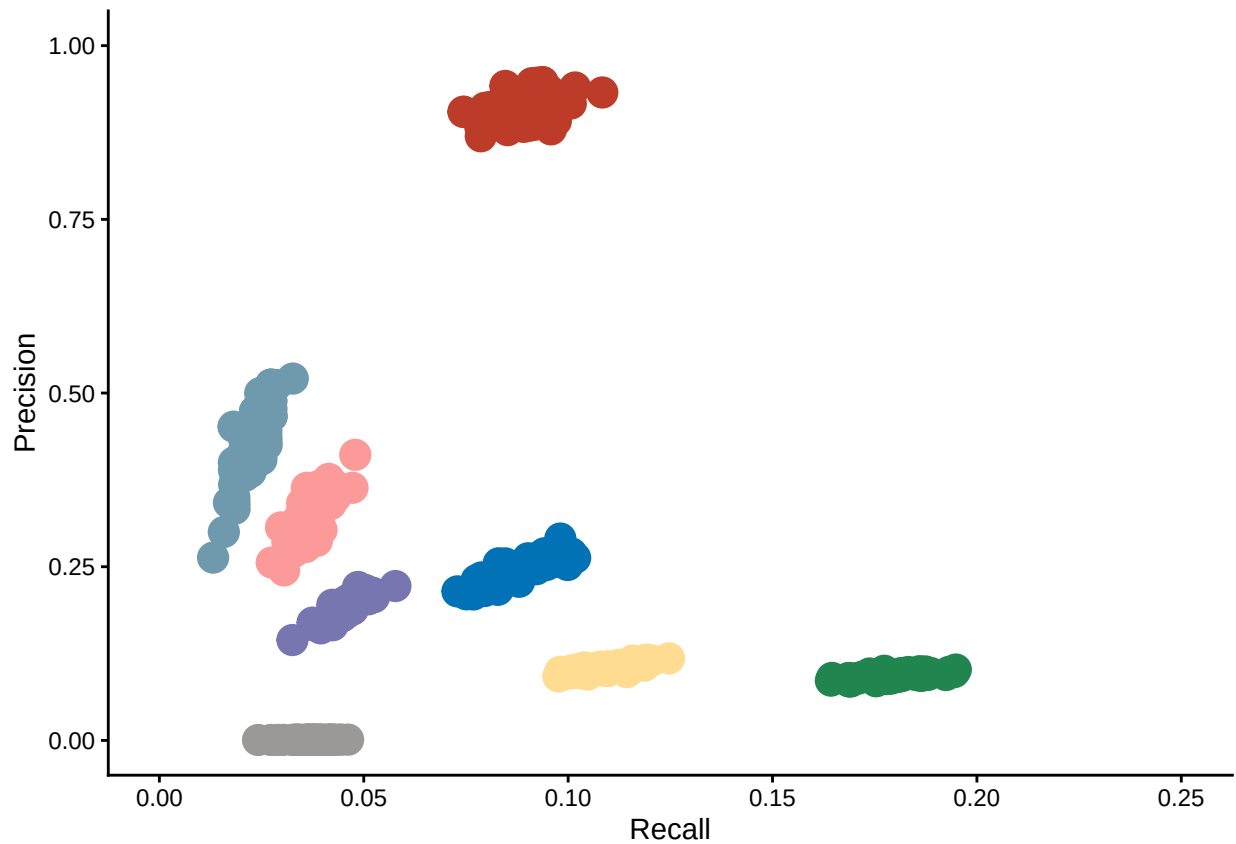
```

p_cSCC_point <- ggplot(data = data_benchmarking_cSCC_res) +
  geom_point(mapping = aes(x = Recall, y = Precision, color = Tool), size = 5, show.legend = FALSE) +
  scale_color_manual(values = colorAssign_1) +
  xlab("Recall") + ylab("Precision") +
  xlim(0, 0.25) +
  ylim(0, 1) +
  theme_classic()

p_cSCC_box <- ggplot(data = data_benchmarking_cSCC_res, aes(x = Tool, y = F1Score)) +
  stat_summary(mapping=aes(fill = Tool),fun=mean, geom = "bar", fun.args = list(mult=1),width=0.7) +
  stat_summary(fun.data=mean_sdl,fun.args = list(mult=1), geom="errorbar",width=0.2) +
  scale_fill_manual(values = colorAssign_1) +
  labs(x = "",y = "F-score") +
  scale_x_discrete(guide = guide_axis(angle = 45)) +
  theme_classic() + guides(fill=guide_legend(title="Method"))

p_cSCC_point

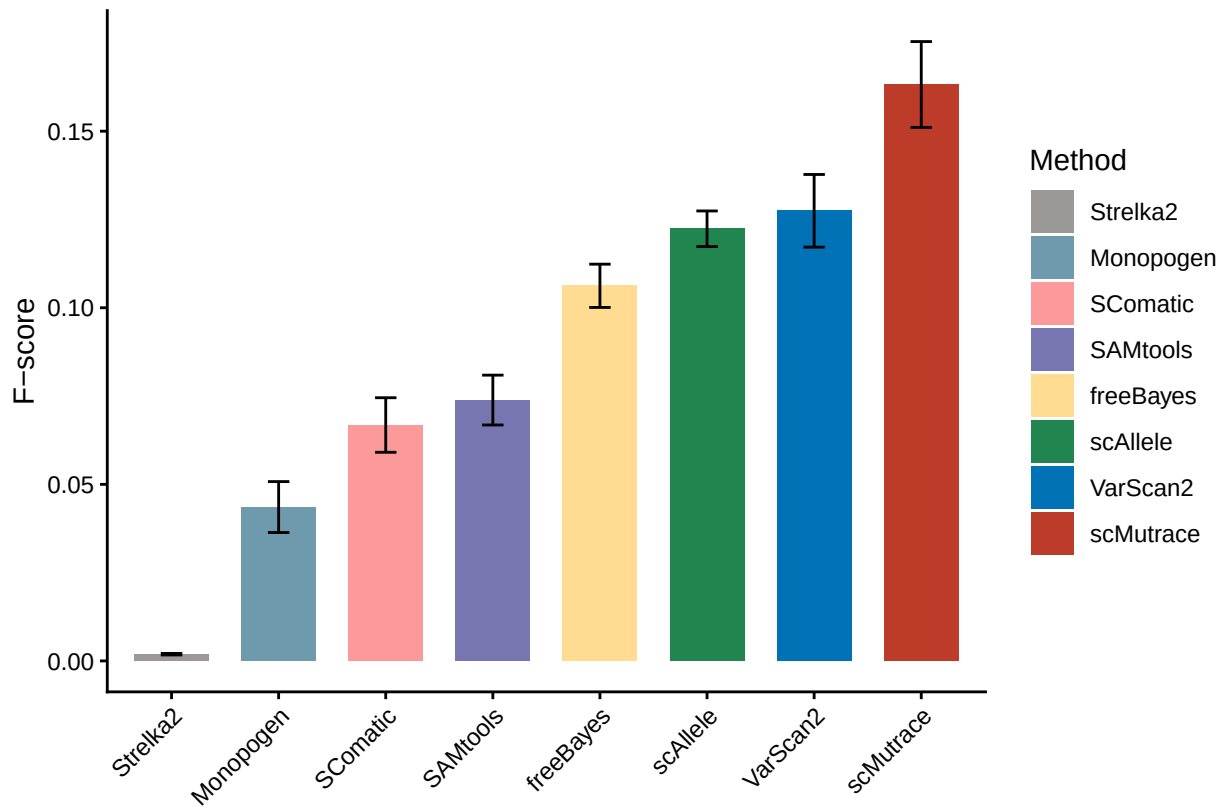
```



```

p_cSCC_box

```

```
# overlap
cSCC_overlap_list <- list()
for (name in names(cSCC_Data_benchmarking)){
  data_tmp <- cSCC_Data_benchmarking[[name]] %>% dplyr::filter(!is.na(chrom_RNA))
  ids_tmp <- unique(data_tmp$id)
  cSCC_overlap_list[[name]] <- ids_tmp
}

# For Strelka2, there are too many unique calls and we can't filter them
upset(fromList(cSCC_overlap_list),
      nsets = length(cSCC_overlap_list),
      order.by = "freq",
      main.bar.color = "steelblue",
      sets.bar.color = "gray23",
      text.scale = c(2, 2, 2, 1.5, 2, 1.5))

## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the UpSetR package.
## Please report the issue to the authors.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
```



```

if (na.rm) {
  keep <- !(is.na(a) | is.na(b))
  a <- a[keep]
  b <- b[keep]
}

denom <- sqrt(sum(a^2)) * sqrt(sum(b^2))
if (denom == 0) return(as.numeric(NA))

sim <- sum(a * b) / denom

sim <- max(min(sim, 1), -1)

return(as.numeric(sim))
}

cSCC_sig <- read.table("../Data/HumancSCC/Mutational_Matrix_SBS_cSCC.txt", header = T, sep = "\t")
cSCC_contribution_raw <- read.csv("../Data/HumancSCC/Signature_contributions_cSCC.csv", header = T, row
cSCC_contribution <- cSCC_contribution_raw[,1]

# calsim
cs_Monopogen_WES <- cosine_similarity(cSCC_sig$Monopogen, cSCC_sig$WES)
cs_SAMtools_WES <- cosine_similarity(cSCC_sig$SAMtools, cSCC_sig$WES)
cs_Somatic_WES <- cosine_similarity(cSCC_sig$Somatic, cSCC_sig$WES)
cs_Strelka2_WES <- cosine_similarity(cSCC_sig$Strelka2, cSCC_sig$WES)
cs_VarScan2_WES <- cosine_similarity(cSCC_sig$VarScan2, cSCC_sig$WES)
cs_freebayes_WES <- cosine_similarity(cSCC_sig$freebayes, cSCC_sig$WES)
cs_scAllele_WES <- cosine_similarity(cSCC_sig$scAllele, cSCC_sig$WES)
cs_scMutrace_WES <- cosine_similarity(cSCC_sig$scMutrace, cSCC_sig$WES)

cSCC_cossim_tools <- c("Monopogen", "SAMtools", "Somatic", "Strelka2", "VarScan2",
                      "freebayes", "scAllele", "scMutrace", "WES")

cSCC_data_cosSim_res <- as.data.frame(matrix(data = 0, nrow = length(cSCC_cossim_tools),
                                             ncol = length(cSCC_cossim_tools)))
rownames(cSCC_data_cosSim_res) <- cSCC_cossim_tools
colnames(cSCC_data_cosSim_res) <- cSCC_cossim_tools

for (i in 1:nrow(cSCC_data_cosSim_res)){
  row_item_tmp <- rownames(cSCC_data_cosSim_res)[i]
  for (j in 1:ncol(cSCC_data_cosSim_res)){
    col_item_tmp <- colnames(cSCC_data_cosSim_res)[j]
    score_tmp <- cosine_similarity(cSCC_sig[,row_item_tmp], cSCC_sig[,col_item_tmp])
    cSCC_data_cosSim_res[i,j] <- score_tmp
  }
}

col_fun_v1 <- colorRamp2(c(0, 0.33, 0.66, 1), c("#404040", "#bababa", "#f4a582", "#ca0020"))

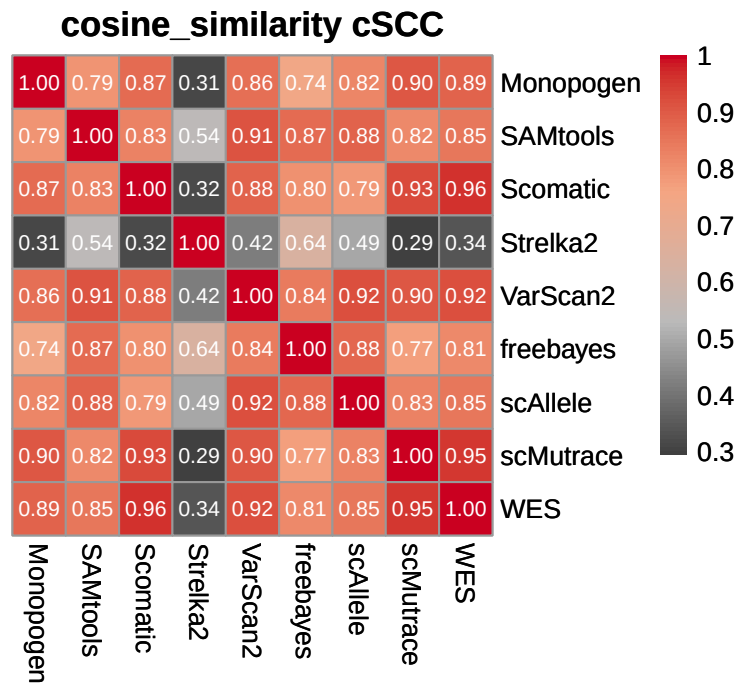
p_cSCC_data_cosSim <- pheatmap(cSCC_data_cosSim_res,
                              cellwidth = 20, cellheight = 20,
                              cluster_rows = F, cluster_cols = F,
                              display_numbers = TRUE, number_color = "white",

```

```

p_cSCC_data_cosSim
    main = "cosine_similarity cSCC",
    color = col_fun_v1(seq(0, 1, length.out = 100)))

```



```

cSCC_contribution_long <- melt(as.matrix(cSCC_contribution))
colnames(cSCC_contribution_long) <- c("Signature", "Tool", "Value")

sbs_color <- c(
  "SBS1" = "#a6cee3", "SBS3" = "#d9d9d9", "SBS5" = "#1f78b4",
  "SBS7a" = "#e31a1c", "SBS7b" = "#fb9a99", "SBS11" = "#bdbdbd",
  "SBS19" = "#b15928", "SBS40b" = "#737373", "SBS43" = "#525252",
  "SBS46" = "#f0f0f0", "SBS58" = "#e5f5e0"
)

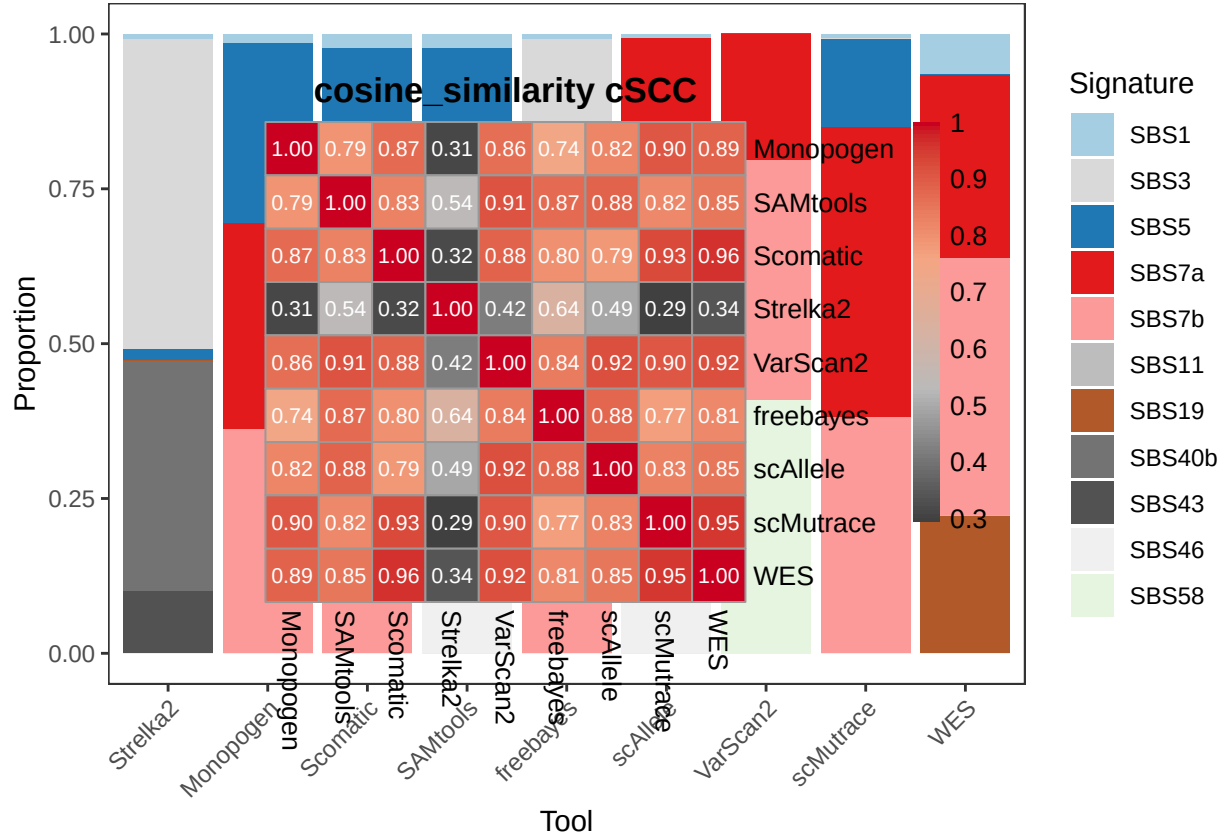
cSCC_contribution_long$Tool <- factor(cSCC_contribution_long$Tool, levels = c("Strelka2", "Monopogen", "SAMtools", "Somatic", "VarScan2", "freebayes", "scAllele", "scMutrace", "WES"))

p_cSCC_contribution <- ggplot(cSCC_contribution_long, aes(x = Tool, y = Value, fill = Signature)) +
  geom_bar(stat = "identity") +
  scale_fill_manual(values = sbs_color) +
  theme_bw() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid = element_blank()
  ) +

```

```
labs(x = "Tool", y = "Proportion", fill = "Signature")
```

```
p_cSCC_contribution  
p_cSCC_data_cosSim
```



7 Mouse scATAC

```
Tools_benchmarking_Mouse <- c("freebayes", "SAMtools", "scAllele",  
                              "scMutrace", "Strelka2", "VarScan2")
```

```
SampleIDs_Mouse <- c("EPSRC1", "EPSRC2", "EPSRC4")
```

```
Mouse_WGS <- readRDS("../Data/MouseEpithelioidSarcoma/GermlineMutation_results_Mouse_WGS_scATAC.rds")  
Mouse_scATA <- readRDS("../Data/MouseEpithelioidSarcoma/GermlineMutation_results_Mouse_scATAC.rds")
```

```
col_benchmarking_Mouse <- c("Iteration", "sampleID", "Tool", "Precision", "Recall", "F1Score")
```

```
data_benchmarking_Mouse_scATA_res <- as.data.frame(matrix(data = 0, nrow = iteration_times * length(Tools_benchmarking_Mouse),  
                                                         ncol = length(col_benchmarking_Mouse)))
```

```
colnames(data_benchmarking_Mouse_scATA_res) <- col_benchmarking_Mouse
```

```

Mouse_Data_scATAC_benchmarking <- list()

for (sampleID in SampleIDs_Mouse){
  mouse_wgs_sample_ini <- data.frame()
  mouse_wgs_sample <- Mouse_WGS[[sampleID]]
  mouse_wgs_sample$id <- paste(mouse_wgs_sample$chrom, mouse_wgs_sample$pos_start,
                               mouse_wgs_sample$ref, sep = "_")
  colnames(mouse_wgs_sample) <- c("chrom_wgs", "pos_start_wgs", "pos_end_wgs",
                                   "ref_wgs", "end_wgs", "Tool_wgs", "id")
  for (tool in Tools_benchmarking_Mouse){
    mouse_ata_sample_tool <- Mouse_scATA[[sampleID]] %>% filter(Tool == tool)
    mouse_ata_sample_tool$id <- paste(mouse_ata_sample_tool$chrom, mouse_ata_sample_tool$pos_start,
                                       mouse_ata_sample_tool$ref, sep = "_")
    colnames(mouse_ata_sample_tool) <- c("chrom_ata", "pos_start_ata", "pos_end_ata",
                                           "ref_ata", "end_ata", "Tool_ata", "id")

    mouse_wgs_ata_tmp <- merge(mouse_wgs_sample, mouse_ata_sample_tool, by = "id", all = T)
    mouse_wgs_ata_tmp$Tool_ata <- tool
    mouse_wgs_sample_ini <- rbind(mouse_wgs_sample_ini, mouse_wgs_ata_tmp)
  }
  Mouse_Data_scATAC_benchmarking[[sampleID]] <- mouse_wgs_sample_ini
}

set.seed(2025)
line_num <- 0
for (iter in 1:iteration_times){
  for (sampleID in SampleIDs_Mouse){
    Mouse_Data_scATAC_benchmarking_bm <- Mouse_Data_scATAC_benchmarking[[sampleID]] %>% sample_frac(sampleID)
    for (tool in Tools_benchmarking_Mouse){
      line_num <- line_num + 1
      Mouse_Data_scATAC_benchmarking_bm_tool <- Mouse_Data_scATAC_benchmarking_bm %>% filter(Tool_ata == tool)
      total_calls <- sum(!is.na(Mouse_Data_scATAC_benchmarking_bm_tool$chrom_ata))
      total_truth <- sum(!is.na(Mouse_Data_scATAC_benchmarking_bm$chrom_wgs))
      tp <- nrow(na.omit(Mouse_Data_scATAC_benchmarking_bm_tool))
      fp <- total_calls - tp
      fn <- total_truth - tp
      sensitivity <- ifelse(total_truth > 0, tp / (tp + fn), NA)
      precision <- ifelse(total_calls > 0, tp / (tp + fp), NA)
      f1 <- ifelse(!is.na(sensitivity) & !is.na(precision) &
                   (sensitivity + precision) > 0,
                   2 * (precision * sensitivity) / (precision + sensitivity),
                   NA)

      data_benchmarking_Mouse_scATA_res[line_num, "Iteration"] <- iter
      data_benchmarking_Mouse_scATA_res[line_num, "sampleID"] <- sampleID
      data_benchmarking_Mouse_scATA_res[line_num, "Tool"] <- tool
      data_benchmarking_Mouse_scATA_res[line_num, "Precision"] <- precision
      data_benchmarking_Mouse_scATA_res[line_num, "Recall"] <- sensitivity
      data_benchmarking_Mouse_scATA_res[line_num, "F1Score"] <- f1
    }
  }
}

colorAssign_1 = c(freebayes = "#FFDC91FF", SAMtools = "#7876B1FF",

```

```

scAllele = "#20854EFF", VarScan2 = "#0072B5FF",
strelka2= "#9B9898", SComatic = "#fb9a99",
Monopogen = "#6F99ADFF", scMutrace = "#BC3C29FF")

data_benchmarking_Mouse_scATA_res <- data_benchmarking_Mouse_scATA_res %>%
  mutate(Tool = fct_reorder(Tool, F1Score, .fun = mean, .na_rm = TRUE))

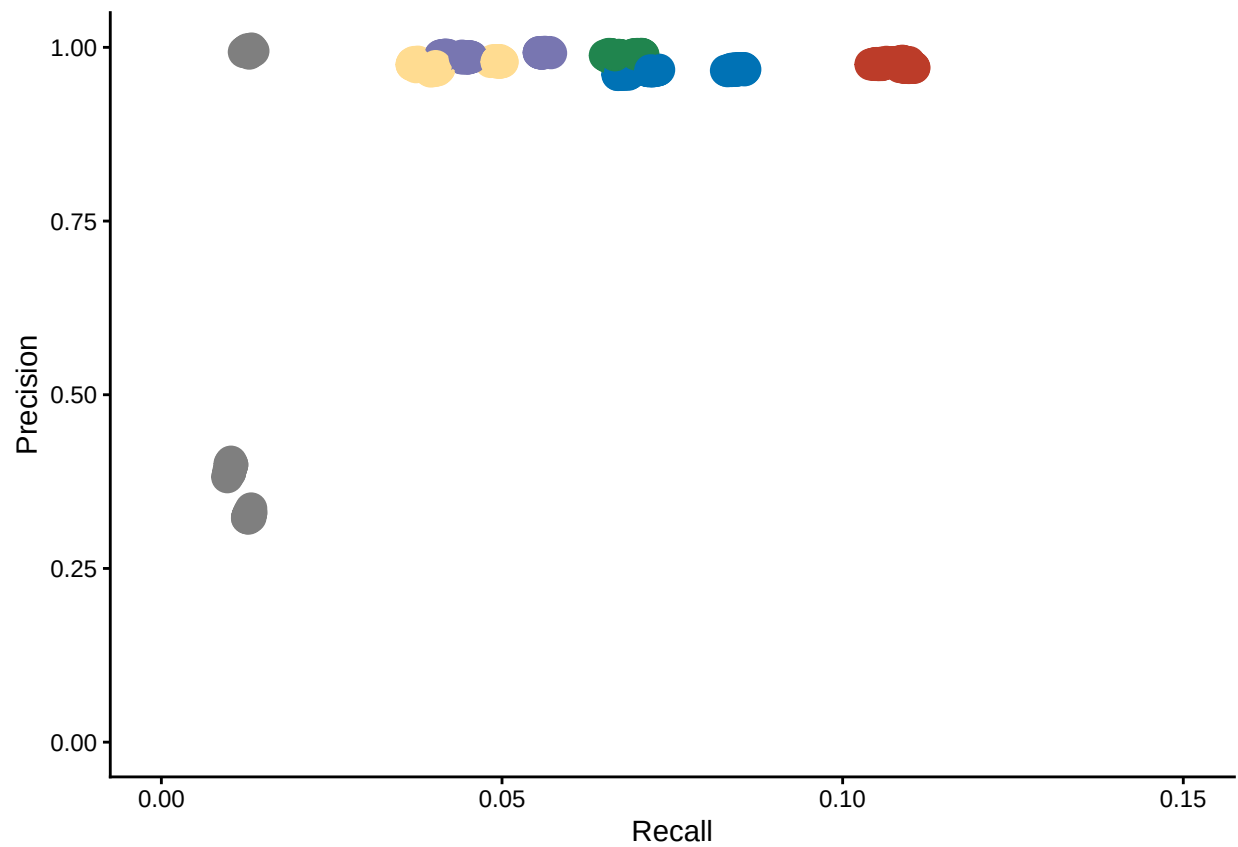
data_benchmarking_Mouse_scATA_res <- na.omit(data_benchmarking_Mouse_scATA_res)

p_Mouse_scATAC_point <- ggplot(data = data_benchmarking_Mouse_scATA_res) +
  geom_point(mapping = aes(x = Recall, y = Precision, color = Tool), size = 5, show.legend = FALSE) +
  scale_color_manual(values = colorAssign_1) +
  xlab("Recall") + ylab("Precision") +
  xlim(0, 0.15) +
  ylim(0, 1) +
  theme_classic()

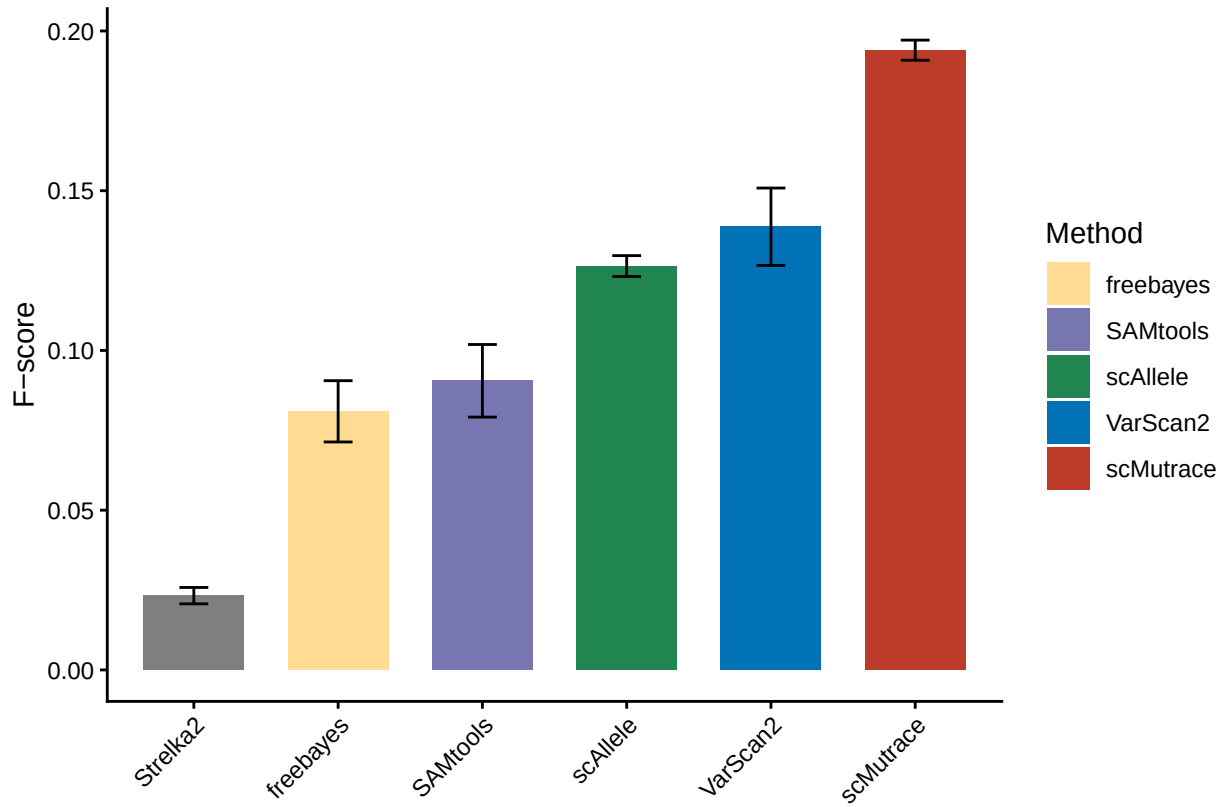
p_Mouse_scATAC_box <- ggplot(data = data_benchmarking_Mouse_scATA_res, aes(x = Tool, y = F1Score)) +
  stat_summary(mapping=aes(fill = Tool),fun=mean, geom = "bar", fun.args = list(mult=1),width=0.7) +
  stat_summary(fun.data=mean_sdl,fun.args = list(mult=1), geom="errorbar",width=0.2) +
  scale_fill_manual(values = colorAssign_1) +
  labs(x = "",y = "F-score") +
  scale_x_discrete(guide = guide_axis(angle = 45)) +
  theme_classic() + guides(fill=guide_legend(title="Method"))

p_Mouse_scATAC_point

```



p_Mouse_scATAC_box



8 Mouse scRNA

```
Tools_benchmarking_Mouse <- c("freebayes", "SAMtools", "scAllele",
                              "scMutrace", "Strelka2", "VarScan2")

SampleIDs_Mouse <- c("EPSRC1", "EPSRC2", "EPSRC4")

Mouse_WGS <- readRDS("../Data/MouseEpithelioidSarcoma/GermlineMutation_results_Mouse_WGS_scRNA.rds")
Mouse_scRNA <- readRDS("../Data/MouseEpithelioidSarcoma/GermlineMutation_results_Mouse_scRNA.rds")

col_benchmarking_Mouse <- c("Iteration", "sampleID", "Tool", "Precision", "Recall", "F1Score")

data_benchmarking_Mouse_scRNA_res <- as.data.frame(matrix(data = 0, nrow = iteration_times * length(Tools_benchmarking_Mouse),
                                                         ncol = length(col_benchmarking_Mouse)))

colnames(data_benchmarking_Mouse_scRNA_res) <- col_benchmarking_Mouse

Mouse_Data_scRNA_benchmarking <- list()

for (sampleID in SampleIDs_Mouse){
  mouse_wgs_sample_ini <- data.frame()
  mouse_wgs_sample <- Mouse_WGS[[sampleID]]
  mouse_wgs_sample$id <- paste(mouse_wgs_sample$chrom, mouse_wgs_sample$pos_start,
```

```

        mouse_wgs_sample$ref, sep = "_")
colnames(mouse_wgs_sample) <- c("chrom_wgs", "pos_start_wgs", "pos_end_wgs",
                                "ref_wgs", "end_wgs", "Tool_wgs", "id")
for (tool in Tools_benchmarking_Mouse){
  mouse_rna_sample_tool <- Mouse_scRNA[[sampleID]] %>% filter(Tool == tool)
  mouse_rna_sample_tool$id <- paste(mouse_rna_sample_tool$chrom, mouse_rna_sample_tool$pos_start,
                                   mouse_rna_sample_tool$ref, sep = "_")
  colnames(mouse_rna_sample_tool) <- c("chrom_ata", "pos_start_ata", "pos_end_ata",
                                       "ref_ata", "end_ata", "Tool_ata", "id")

  mouse_wgs_rna_tmp <- merge(mouse_wgs_sample, mouse_rna_sample_tool, by = "id", all = T)
  mouse_wgs_rna_tmp$Tool_ata <- tool
  mouse_wgs_sample_ini <- rbind(mouse_wgs_sample_ini, mouse_wgs_rna_tmp)
}
Mouse_Data_scRNA_benchmarking[[sampleID]] <- mouse_wgs_sample_ini
}

set.seed(2025)
line_num <- 0
for (iter in 1:iteration_times){
  for (sampleID in SampleIDs_Mouse){
    Mouse_Data_scRNA_benchmarking_bm <- Mouse_Data_scRNA_benchmarking[[sampleID]] %>% sample_frac(sampleID)
    for (tool in Tools_benchmarking_Mouse){
      line_num <- line_num + 1
      Mouse_Data_scRNA_benchmarking_bm_tool <- Mouse_Data_scRNA_benchmarking_bm %>% filter(Tool_ata == tool)
      total_calls <- sum(!is.na(Mouse_Data_scRNA_benchmarking_bm_tool$chrom_ata))
      total_truth <- sum(!is.na(Mouse_Data_scRNA_benchmarking_bm$chrom_wgs))
      tp <- nrow(na.omit(Mouse_Data_scRNA_benchmarking_bm_tool))
      fp <- total_calls - tp
      fn <- total_truth - tp
      sensitivity <- ifelse(total_truth > 0, tp / (tp + fn), NA)
      precision <- ifelse(total_calls > 0, tp / (tp + fp), NA)
      f1 <- ifelse(!is.na(sensitivity) & !is.na(precision) &
                  (sensitivity + precision) > 0,
                  2 * (precision * sensitivity) / (precision + sensitivity),
                  NA)

      data_benchmarking_Mouse_scRNA_res[line_num, "Iteration"] <- iter
      data_benchmarking_Mouse_scRNA_res[line_num, "sampleID"] <- sampleID
      data_benchmarking_Mouse_scRNA_res[line_num, "Tool"] <- tool
      data_benchmarking_Mouse_scRNA_res[line_num, "Precision"] <- precision
      data_benchmarking_Mouse_scRNA_res[line_num, "Recall"] <- sensitivity
      data_benchmarking_Mouse_scRNA_res[line_num, "F1Score"] <- f1
    }
  }
}

colorAssign_1 = c(freebayes = "#FFDC91FF", SAMtools = "#7876B1FF",
                  scAllele = "#20854EFF", VarScan2 = "#0072B5FF",
                  strelka2 = "#9B9898", SComatic = "#fb9a99",
                  Monopogen = "#6F99ADFF", scMutrace = "#BC3C29FF")

data_benchmarking_Mouse_scRNA_res <- data_benchmarking_Mouse_scRNA_res %>%
  mutate(Tool = fct_reorder(Tool, F1Score, .fun = mean, .na_rm = TRUE))

```

```

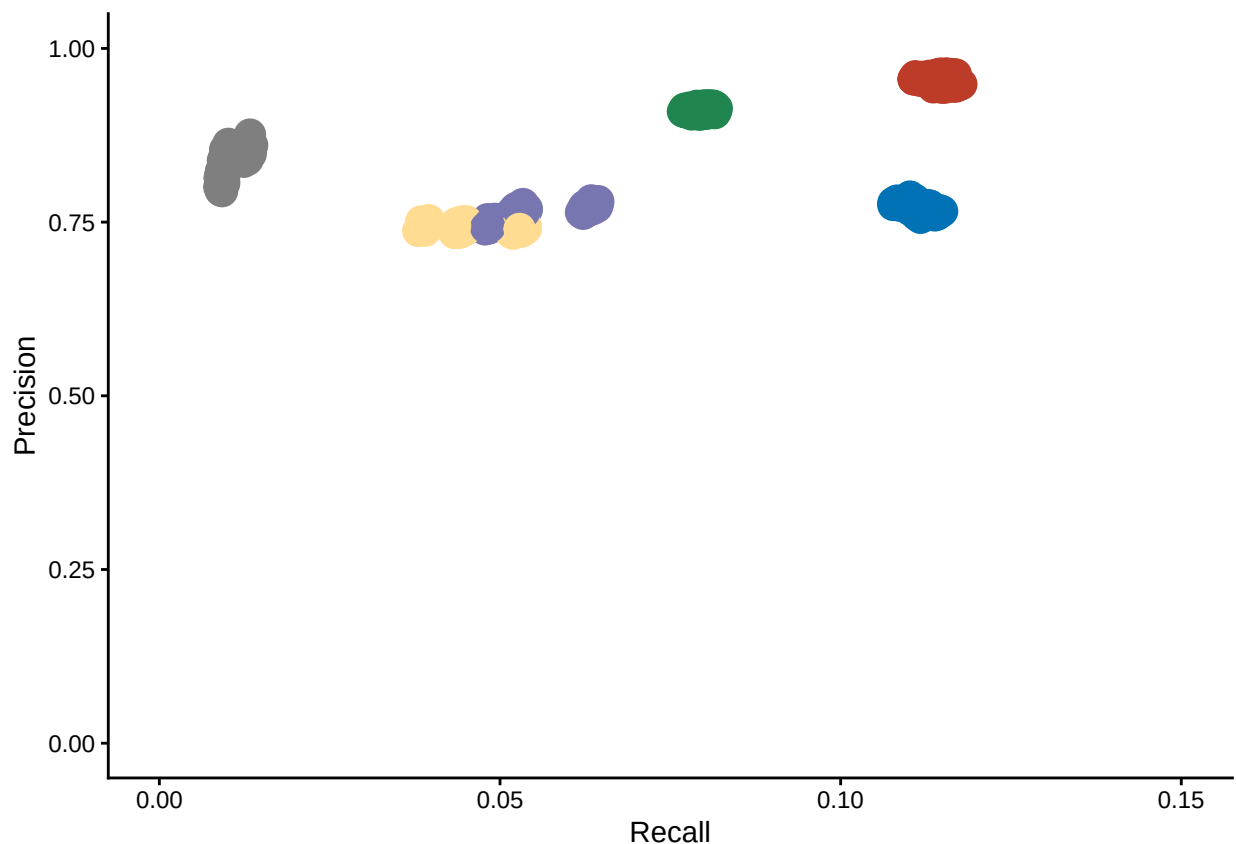
data_benchmarking_Mouse_scrNA_res <- na.omit(data_benchmarking_Mouse_scrNA_res)

p_Mouse_scrNA_point <- ggplot(data = data_benchmarking_Mouse_scrNA_res) +
  geom_point(mapping = aes(x = Recall, y = Precision, color = Tool), size = 5, show.legend = FALSE) +
  scale_color_manual(values = colorAssign_1) +
  xlab("Recall") + ylab("Precision") +
  xlim(0, 0.15) +
  ylim(0, 1) +
  theme_classic()

p_Mouse_scrNA_box <- ggplot(data = data_benchmarking_Mouse_scrNA_res, aes(x = Tool, y = F1Score)) +
  stat_summary(mapping=aes(fill = Tool),fun=mean, geom = "bar", fun.args = list(mult=1),width=0.7) +
  stat_summary(fun.data=mean_sdl,fun.args = list(mult=1), geom="errorbar",width=0.2) +
  scale_fill_manual(values = colorAssign_1) +
  labs(x = "",y = "F-score") +
  scale_x_discrete(guide = guide_axis(angle = 45)) +
  theme_classic() + guides(fill=guide_legend(title="Method"))

p_Mouse_scrNA_point

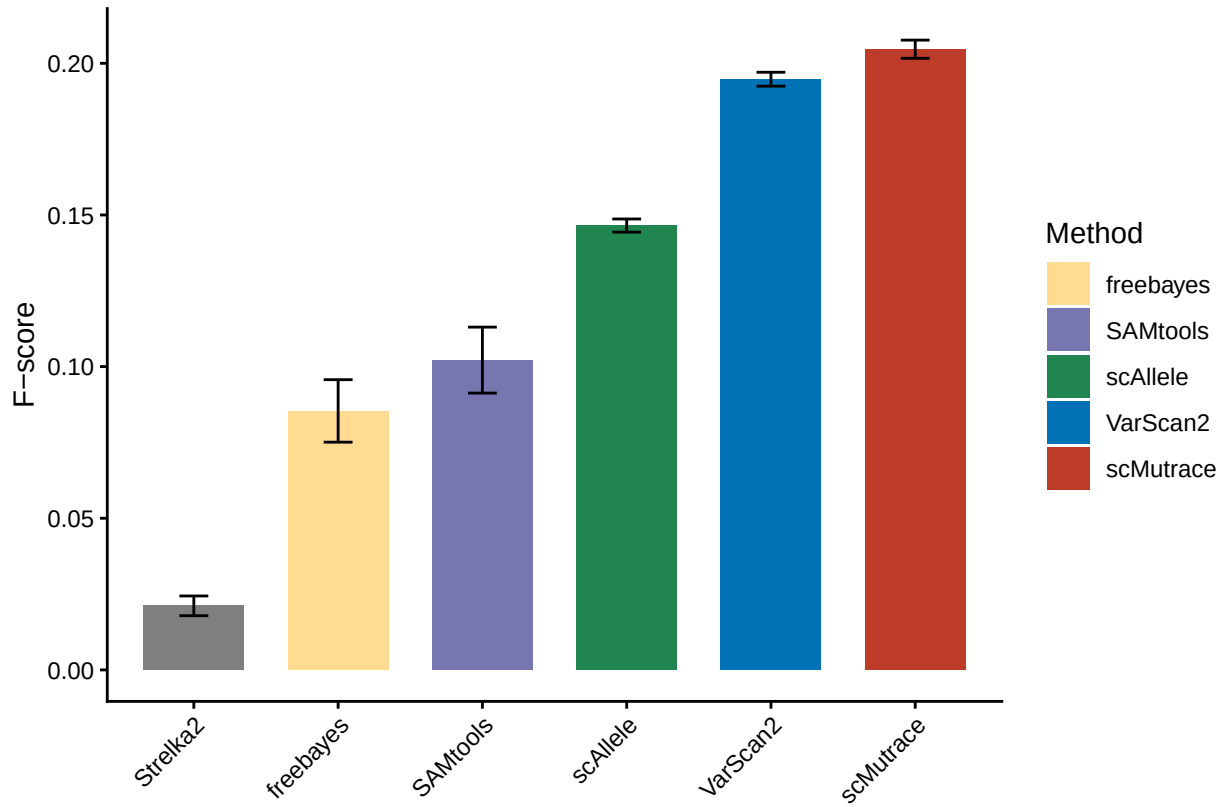
```



```

p_Mouse_scrNA_box

```



9 Mouse scRNA scATA calConsistencyscore

```

SampleIDs_Mouse <- c("EPSRC1", "EPSRC2", "EPSRC4")
Mouse_Consistencyscore <- readRDS("../Data/MouseEpithelioidSarcoma/GermlineMutation_results_Mouse_Consi

Mouse_consistencyScoreMatrix_RNA_ATA <- matrix(nrow = length(SampleIDs_Mouse), ncol = length(SampleIDs_M
colnames(Mouse_consistencyScoreMatrix_RNA_ATA) <- SampleIDs_Mouse
rownames(Mouse_consistencyScoreMatrix_RNA_ATA) <- SampleIDs_Mouse

for (i in 1:nrow(Mouse_consistencyScoreMatrix_RNA_ATA)){
  sampleID_i <- SampleIDs_Mouse[i]
  rna_tmp <- Mouse_Consistencyscore[[sampleID_i]] %>% filter(sequencingType == "scRNA")
  rna_tmp$id <- paste(rna_tmp$chrom, rna_tmp$pos_start, rna_tmp$ref, sep = "_")
  rna_tmp_id <- rna_tmp$id

  for (j in 1:ncol(Mouse_consistencyScoreMatrix_RNA_ATA)){
    sampleID_j <- SampleIDs_Mouse[j]
    ata_tmp <- Mouse_Consistencyscore[[sampleID_j]] %>% filter(sequencingType == "scATA")
    ata_tmp$id <- paste(ata_tmp$chrom, ata_tmp$pos_start, ata_tmp$ref, sep = "_")
    ata_tmp_id <- ata_tmp$id

    inter_len <- length(intersect(rna_tmp_id, ata_tmp_id))
    union_len <- length(union(rna_tmp_id, ata_tmp_id))
  }
}

```

```

score_tmp <- ifelse(union_len == 0, NA, inter_len / union_len)

  Mouse_consistencyScoreMatrix_RNA_ATA[i, j] <- score_tmp
}
}

rownames(Mouse_consistencyScoreMatrix_RNA_ATA) <- paste0(SampleIDs_Mouse, "_scMutrace_scRNA")
colnames(Mouse_consistencyScoreMatrix_RNA_ATA) <- paste0(SampleIDs_Mouse, "_scMutrace_scATA")

max_val <- max(Mouse_consistencyScoreMatrix_RNA_ATA)

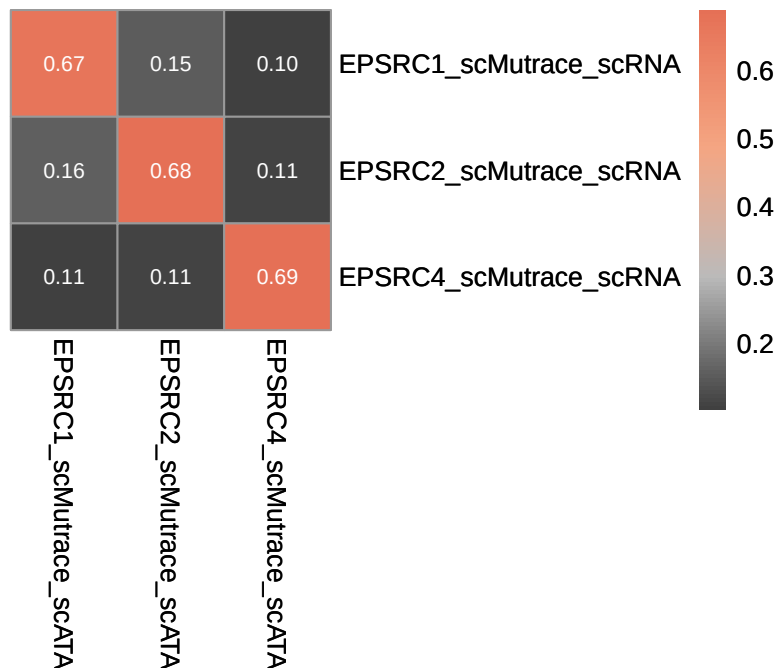
col_fun_v1 <- colorRamp2(c(0, max_val * 0.33, max_val * 0.66, 1), c("#404040", "#bababa", "#f4a582", "#f4a582"))
col_vec <- col_fun_v1(seq(0, max_val, length.out = 100))

p_consistencyScoreMatrix_RNA_ATA <- pheatmap(Mouse_consistencyScoreMatrix_RNA_ATA,
  cellwidth = 40, cellheight = 40,
  cluster_rows = F, cluster_cols = F,
  display_numbers = TRUE, number_color = "white",
  main = "Consistency score of scRNA and scATAC",
  color = col_vec)

p_consistencyScoreMatrix_RNA_ATA

```

Consistency score of scRNA and scATAC



10 BrainPFC

```
# common region with at least 2 cells
PFC_Mutation <- readRDS("../Data/HumanPFC/BrainPFC_Mutation_res.rds")

PFC_sampleIDs <- c("Sample018", "Sample019", "Sample020", "Sample021", "Sample022",
                  "Sample023", "Sample025", "Sample026", "Sample027", "Sample028",
                  "Sample029", "Sample030", "Sample031", "Sample032", "Sample033")

consistencyScoreMatrix_RNA_ATA_PFC <- matrix(nrow = length(PFC_sampleIDs), ncol = length(PFC_sampleIDs))

for (i in 1:nrow(consistencyScoreMatrix_RNA_ATA_PFC)){
  # sampleID
  sample_RNA_id <- paste0(PFC_sampleIDs[i], "_RNA")
  # data
  sample_RNA_data <- PFC_Mutation[[sample_RNA_id]]
  sample_RNA_data$tagIndex <- paste0(sample_RNA_data$V1, "_", sample_RNA_data$V2)
  tagIndex_RNA <- sample_RNA_data$tagIndex
  for (j in 1:ncol(consistencyScoreMatrix_RNA_ATA_PFC)){
    sample_ATA_id <- paste0(PFC_sampleIDs[j], "_ATA")
    sample_ATA_data <- PFC_Mutation[[sample_ATA_id]]
    sample_ATA_data$tagIndex <- paste0(sample_ATA_data$V1, "_", sample_ATA_data$V2)
    tagIndex_ATA <- sample_ATA_data$tagIndex
    consistencyScoreMatrix_RNA_ATA_PFC[i, j] <- length(unique(intersect(tagIndex_RNA, tagIndex_ATA)))/1
    rm(sample_ATA_data)
  }
  rm(sample_RNA_data)
}

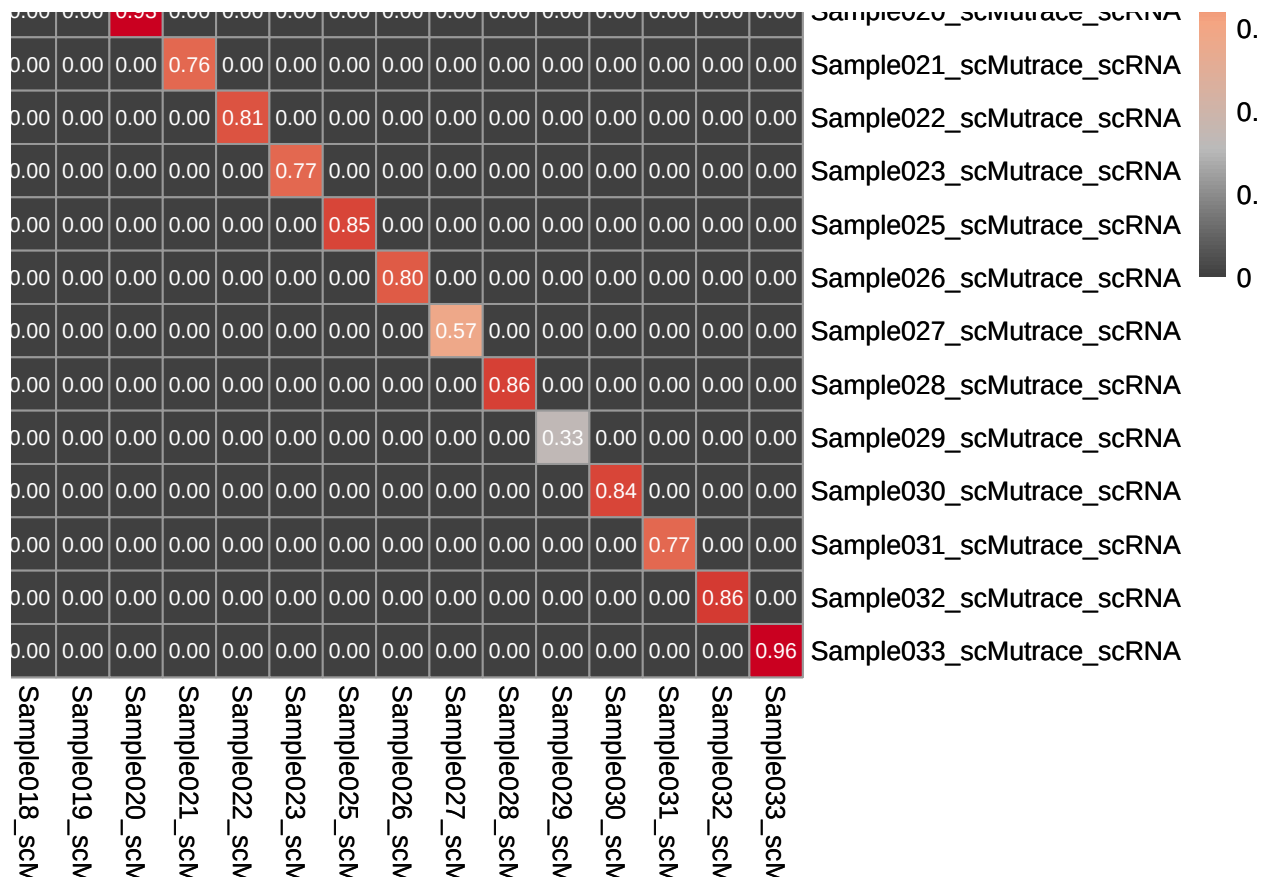
rownames(consistencyScoreMatrix_RNA_ATA_PFC) <- paste0(PFC_sampleIDs, "_scMutrace_scRNA")
colnames(consistencyScoreMatrix_RNA_ATA_PFC) <- paste0(PFC_sampleIDs, "_scMutrace_scATA")

max_val <- max(consistencyScoreMatrix_RNA_ATA_PFC)

col_fun_v1 <- colorRamp2(c(0, max_val * 0.33, max_val * 0.66, max_val), c("#404040", "#bababa", "#f4a585", "#f4a585"))

p_consistencyScoreMatrix_RNA_ATA_brain <- pheatmap(consistencyScoreMatrix_RNA_ATA_PFC,
  cellwidth = 20, cellheight = 20,
  cluster_rows = F, cluster_cols = F,
  display_numbers = TRUE, number_color = "white",
  main = "Consistency score of scRNA and scATAC",
  color = col_fun_v1(seq(0, 1, length.out = 100)))

p_consistencyScoreMatrix_RNA_ATA_brain
```



11 sessionInfo

```

sessionInfo()
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.2
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/Stockholm
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] UpSetR_1.4.0      bedtoolsr_2.30.0-7 gridExtra_2.3      patchwork_1.3.2
## [5] circlize_0.4.16  pheatmap_1.0.13    reshape2_1.4.4     ggplot2_4.0.0

```

```
## [9] forcats_1.0.0      readxl_1.4.5      scales_1.4.0      dplyr_1.1.4
## [13] ggsci_3.2.0
##
## loaded via a namespace (and not attached):
## [1] generics_0.1.4      shape_1.4.6.1      stringi_1.8.7
## [4] digest_0.6.37       magrittr_2.0.4      evaluate_1.0.5
## [7] grid_4.5.1          RColorBrewer_1.1-3 fastmap_1.2.0
## [10] cellranger_1.1.0    plyr_1.8.9          backports_1.5.0
## [13] nnet_7.3-20         Formula_1.2-5       GlobalOptions_0.1.2
## [16] cli_3.6.5           crayon_1.5.3        rlang_1.1.6
## [19] Hmisc_5.2-3         base64enc_0.1-3     withr_3.0.2
## [22] yaml_2.3.10         tools_4.5.1         checkmate_2.3.3
## [25] htmlTable_2.4.3     colorspace_2.1-2    rpart_4.1.24
## [28] vctrs_0.6.5         R6_2.6.1            lifecycle_1.0.4
## [31] stringr_1.5.2       htmlwidgets_1.6.4   foreign_0.8-90
## [34] cluster_2.1.8.1     pkgconfig_2.0.3     pillar_1.11.1
## [37] gtable_0.3.6        data.table_1.17.8   glue_1.8.0
## [40] Rcpp_1.1.0          xfun_0.53           tibble_3.3.0
## [43] tidyselect_1.2.1    rstudioapi_0.17.1   knitr_1.50
## [46] dichromat_2.0-0.1   farver_2.1.2        htmltools_0.5.8.1
## [49] rmarkdown_2.30      labeling_0.4.3      compiler_4.5.1
## [52] S7_0.2.0
```